

PYTHON BISTRO

Project Report

Student Name	Alan S Kumar
Registration Number	26BCE10296
Project Title	Python Bistro
Technology	Python 3

1. Abstract

Python Bistro is a command-line restaurant ordering and billing system developed in Python. The application accepts customer information, supports dine-in and takeaway orders, displays a predefined menu, records item quantities, calculates the subtotal, applies 5% GST, and prints a final bill. The project demonstrates fundamental Python programming concepts through a practical real-world use case.

2. Problem Statement

Restaurants require a simple mechanism to record customer orders and calculate bills accurately. Manual calculations can lead to mistakes in quantities, subtotals, or taxes. Python Bistro provides a lightweight command-line solution that performs these operations programmatically while validating user input.

3. Objectives

- Accept customer and order-type information.
- Display a predefined menu.
- Record multiple items and quantities.
- Calculate subtotal and 5% GST.
- Generate a final formatted bill.
- Validate invalid input and handle errors.
- Demonstrate modular Python programming and automated testing.

4. Technologies and Concepts

The project uses Python 3 without external libraries. Major concepts include functions, dictionaries, loops, conditional statements, string formatting, exception handling, modular programming, and assertions/testing.

5. System Modules

1. Customer and order-type management
2. Menu display and item selection
3. Current-order management
4. Billing and GST calculation
5. Input validation and error handling
6. Automated calculation testing

6. Functional Requirements

The system shall accept a valid customer name; allow selection of Dine-In or Takeaway; display available menu items; accept item numbers and quantities; maintain the current order; calculate subtotal; calculate 5% GST; generate a final bill; and reject invalid inputs without terminating unexpectedly.

7. Non-Functional Requirements

The application should be simple to operate, readable, modular, maintainable, reliable for supported calculations, and executable using a standard Python 3 installation.

8. Billing Algorithm

For every selected item, price is multiplied by quantity and added to the subtotal. GST is calculated as $\text{subtotal} \times 0.05$. The final bill is $\text{subtotal} + \text{GST}$. The same menu item selected multiple times is accumulated into a single quantity.

9. Implementation Details

main.py controls the user interaction and application flow. utils.py stores menu data and reusable billing/printing functions. test_bistro.py verifies the calculation functions independently. The program uses dictionaries for menu and order data and exception handling for invalid numeric input.

10. Testing

The test module covers single-item calculation, multiple-item calculation, GST, final total, empty order handling, and a complete bill calculation. The current execution result is ****ALL TESTS PASSED!****

11. Limitations

The current version uses in-memory data only. Orders are not persisted after the program closes. The interface is command-line based, and the menu is defined directly in the source code. Database, payment, authentication, and online ordering features are outside the current scope.

12. Future Enhancements

Future versions could add persistent storage, customer history, discounts, table management, PDF receipts, database integration, graphical interfaces, online ordering/payment, and administrator-controlled menu management.

13. Conclusion

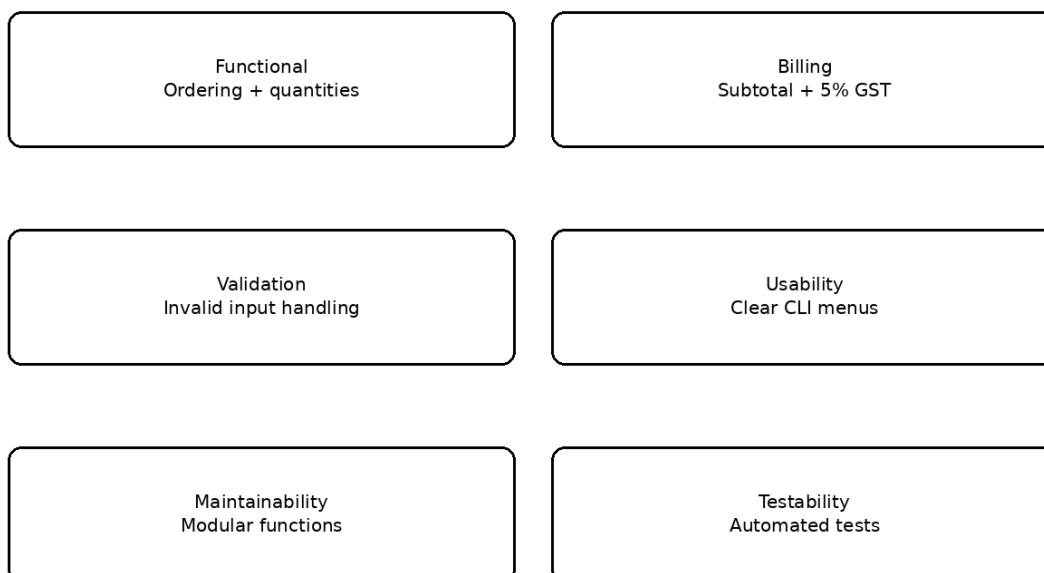
Python Bistro successfully demonstrates how Python can be used to solve a practical restaurant billing problem with a clear modular structure. It combines user interaction, data structures, calculations, validation, and testing into a complete command-line application.

Project Documentation

The following diagrams and evidence are included in the project package.

Requirements

Python Bistro - Requirements



Objectives

Python Bistro - Objectives

1. Accept customer details and order type

2. Display a predefined restaurant menu

3. Record multiple items and quantities

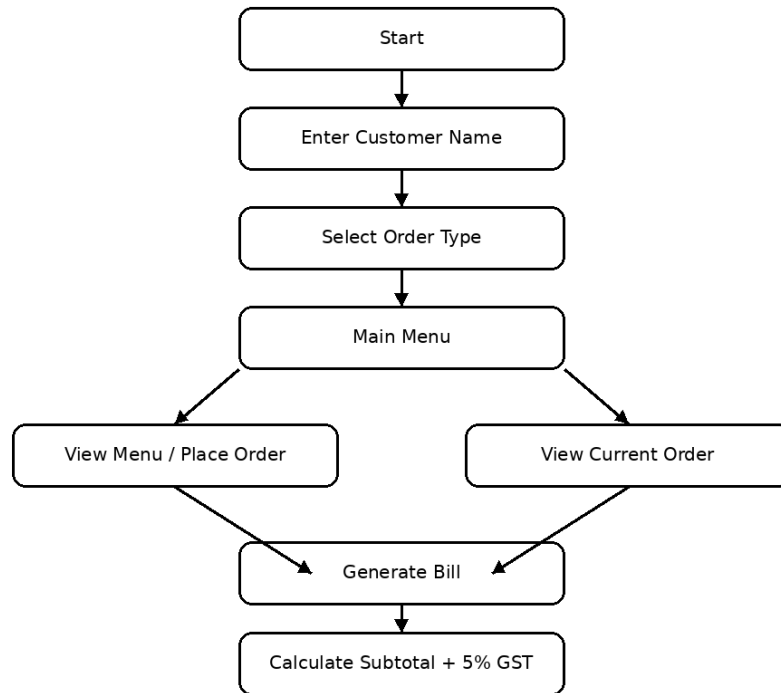
4. Calculate subtotal and 5% GST

5. Generate a clear final bill

6. Demonstrate modular Python and testing

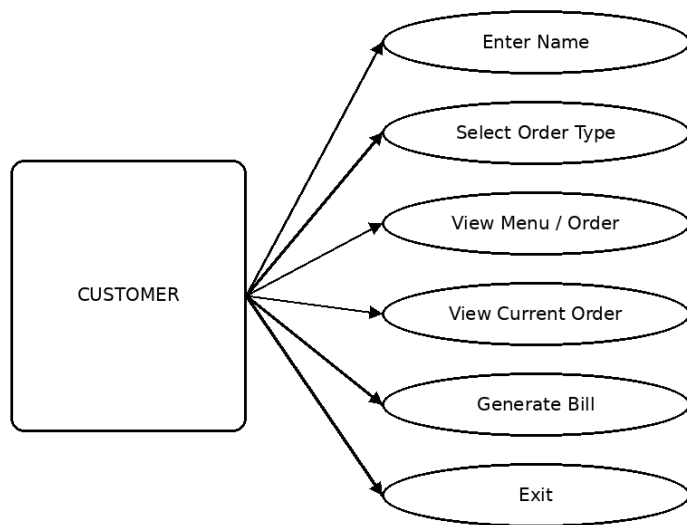
Flowchart

Python Bistro - Flowchart



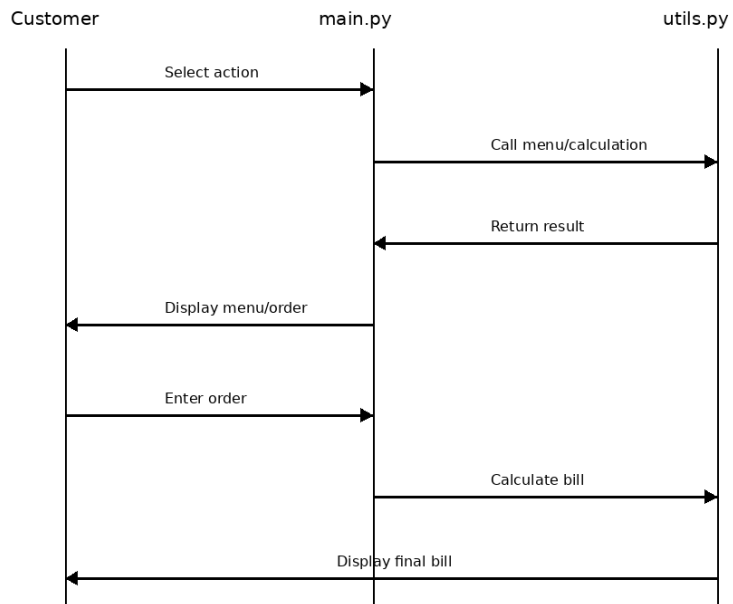
Use Case Diagram

Python Bistro - Use Case Diagram



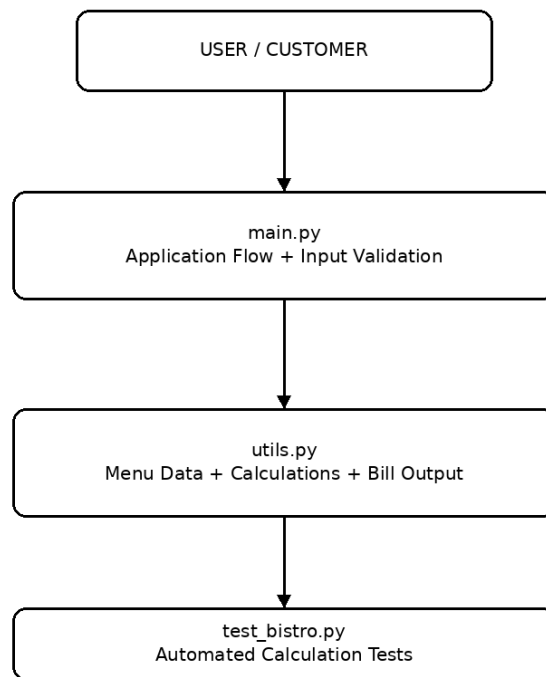
Sequence Diagram

Python Bistro - Sequence Diagram



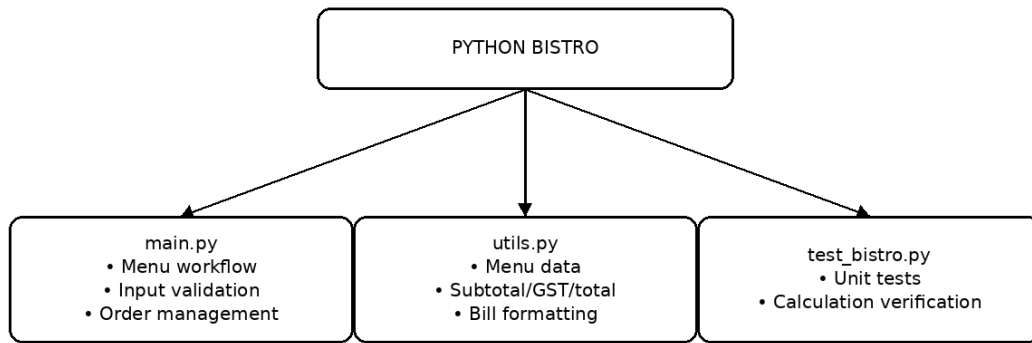
System Architecture

Python Bistro - System Architecture



Component Diagram

Python Bistro - Component Diagram



Sample Output

Python Bistro - Sample Terminal Output

```

WELCOME TO PYTHON BISTRO

Enter your name:
Select Order Type
1. Dine In
2. Takeaway
Enter Choice:
=====
1. View Menu & Place Order
2. View Current Order
3. Generate Bill
4. Exit
=====
Enter your choice:
=====
1. Burger MENU INR 120
2. Pizza INR 180
3. Pasta INR 150
4. French Fries INR 80
5. Sandwich INR 100
6. Cold Coffee INR 70
7. Coca Cola INR 50
8. Coffee INR 25
9. Tea INR 20
10. Thali INR 180
0. Finish Order
=====
Enter item number: Quantity: 2 x Pizza added!

1. Burger MENU INR 120
2. Pizza INR 180
3. Pasta INR 150
4. French Fries INR 80
5. Sandwich INR 100
6. Cold Coffee INR 70
7. Coca Cola INR 50
8. Coffee INR 25
9. Tea INR 20
10. Thali INR 180
0. Finish Order
=====
Enter item number: Quantity: 0 x Burger added!

1. Burger MENU INR 120
2. Pizza INR 180
3. Pasta INR 150
4. French Fries INR 80
5. Sandwich INR 100
6. Cold Coffee INR 70
7. Coca Cola INR 50
8. Coffee INR 25
9. Tea INR 20
10. Thali INR 180
0. Finish Order
=====
Enter item number: Quantity: Please enter a number greater than 0.
Quantity: 2 x Burger added!

1. Burger MENU INR 120
2. Pizza INR 180
3. Pasta INR 150
4. French Fries INR 80
5. Sandwich INR 100
6. Cold Coffee INR 70
7. Coca Cola INR 50
8. Coffee INR 25
9. Tea INR 20
10. Thali INR 180
0. Finish Order
=====
Enter item number: Quantity:
=====
Spreadsheet runtime warning failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.L2H2Y5oc/artifact_tool_v2.2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 70, in <module>
    file = tmp/tmp.L2H2Y5oc/artifact_tool_v2.2.8.22/artifact_tool/spreadsheet_warmup.py", line 749, in warm_collaboration_flow
  File "/tmp/tmp.L2H2Y5oc/artifact_tool_v2.2.8.22/artifact_tool/generated/interface/models.py", line 32
  32, in hydrate_crdt_from_protobuf
  File "/tmp/tmp.L2H2Y5oc/artifact_tool_v2.2.8.22/artifact_tool/rpc/remotepv.py", line 749, in call
  File "/tmp/tmp.L2H2Y5oc/artifact_tool_v2.2.8.22/artifact_tool/rpc/client.py", line 100, in call
ValueError: int() argument must be either a string or a real-valued number, not an empty collaborative document
Traceback (most recent call last):
  File "[35m~/data/pythonbistro/main.py"[0m, line [35m97[0m, in [35m-module[0m[0m]
    [35m-args[0m][1][35m-[0m
    [35mcmd_items[0m][1][35m-(order)[0m]
    [35mcmd_items[0m][0][1][35m-(order)[0m]
  File "[35m~/data/pythonbistro/main.py"[0m, line [35m93[0m[0m, in [35mcmd_items[0m]
  File "[35m~/data/pythonbistro/main.py"[0m, line [35m6[0m[0m, in [35mgot_positive_integer[0m]
ValueError: int() argument must be either a string or a real-valued number, not an empty collaborative document
[35mOFFER[0m[0m]
[35mOFFER[0m]
reading a line[0m

```

Test Results

Python Bistro - Test Results

```
$ python test_bistro.py
```

```
ALL TESTS PASSED!
```

```
Spreadsheet runtime warmup failed during python startup
```

```
Traceback (most recent call last):
```

```
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_d
```

```
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 785,
```

```
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 720,
```

```
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 704,
```